Comprehensive Operating Systems Exam Handbook

Module 1: Basics of Operating Systems

Table of Contents

1. Introduction to the Operating System
2. History and Evolution
3. Types of Operating Systems
4. User Interfaces and System Calls
5. Case Study: The UNIX Philosophy
6. Unit Summary & Cheat Sheet
7. Exam Strategies & Pitfalls
8. Practice Question Bank

1. Introduction to the Operating System

1.1 Strict Definitions

An Operating System (OS) is not just "software that runs the computer." That is a layman's definition that will lose you marks in an exam. A rigorous definition views the OS from two distinct perspectives:

1. **The Resource Allocator View:** The OS manages all resources (CPU time, memory space, file storage, I/O devices). It decides between conflicting requests for efficient and fair resource use.
2. **The Control Program View:** The OS controls the execution of user programs to prevent errors and improper use of the computer. It exists to define the "rules of the road."

The Kernel: The one program running at all times on the computer. Everything else is either a system program (ships with the OS) or an application program.

1.2 Core Goals

- **Convenience:** Making the computer easier to use (primary goal of Windows/macOS).
- **Efficiency:** Using computer resources in an efficient manner (primary goal of Mainframes/Linux servers).
- **Robustness:** Ensuring reliability and fault tolerance.
- **Scalability:** The ability to handle increasing workloads (e.g., adding more CPUs).

1.3 Worked Examples: Resource Allocation

Example 1: CPU Scheduling Logic *Scenario:* Two processes, P1 (video rendering) and P2 (text editor), need the CPU. *OS Decision:* P1 is CPU-intensive; P2 is I/O-intensive (waiting for

keystrokes). *Mechanism:* The OS gives P2 high priority but short time slices. This keeps the UI responsive. P1 gets longer time slices but lower priority. *Result:* Maximized CPU utilization without freezing the mouse cursor.

Example 2: Memory Protection *Scenario:* A buggy student C++ program tries to write data to memory address `0x00000000` (often reserved for the OS kernel). *OS Action:* The hardware Memory Management Unit (MMU) detects the access violation. It raises a trap. The OS catches the trap and kills the process with a "Segmentation Fault." *Result:* The OS survives; the bad program does not.

2. History and Evolution

Understanding history explains *why* modern OSs are complex.

2.1 Generation 1: Vacuum Tubes (1945–1955)

- **Technology:** Plugboards. No OS existed.
- **Mode:** Absolute machine language.
- **Inefficiency:** Users signed up for machine time. If code failed in 10 seconds, the remaining 50 minutes of the slot were wasted.

2.2 Generation 2: Transistors & Batch Systems (1955–1965)

- **Concept: Batch Processing**.
- **Job:** A deck of punch cards.
- **The Monitor:** The first rudimentary OS. It automatically read the next job when the current one finished.
- **JCL (Job Control Language):** Used to tell the monitor about the job (e.g., `$JOB`, `$LOAD`, `$RUN`).

2.3 Generation 3: ICs & Multiprogramming (1965–1980)

- **Spooling (Simultaneous Peripheral Operation On-Line):** Reading jobs from cards to disk while the CPU executes another job.
- **Multiprogramming:** Keeping multiple jobs in memory. When Job A waits for I/O, the OS switches to Job B.
  - *Critical derivation:* CPU Utilization = $1 - p^n$, where p is the fraction of time a process waits for I/O and n is the number of processes in memory.
- **Time-Sharing:** Multiprogramming with a timer interrupt. The CPU switches so fast that users feel they have the machine to themselves.

2.4 Generation 4: Personal Computers (1980–Present)

- **GUI:** Xerox PARC $\rightarrow$ Apple Macintosh $\rightarrow$ Windows.
- **Network OS:** Systems aware of the network (TCP/IP integration).

3. Types of Operating Systems

3.1 Batch Operating System

- **Mechanism:** Jobs are grouped into batches with similar needs. No user interaction.
- **Pro:** High processor utilization.
- **Con:** Difficult to debug; long turnaround time.
- **Application:** Payroll systems, scientific weather forecasting.

3.2 Time-Sharing (Multitasking) OS

- **Mechanism:** Logical extension of multiprogramming. Uses **Round Robin** scheduling.
- **Quantum:** A small unit of time (e.g., 10-100ms) allocated to each process.
- **Application:** Modern desktops, servers.

3.3 Real-Time Operating System (RTOS)

Rigid time requirements.

- **Hard Real-Time:** Critical tasks *must* complete on time. Missing a deadline = total system failure.
  - *Example:* Pacemaker, Missile guidance, ABS brakes.
  - *Storage:* Often no virtual memory; data stored in ROM.
- **Soft Real-Time:** Critical task gets priority, but missing a deadline is annoying, not fatal.
  - *Example:* Video streaming, Virtual Reality.

3.4 Distributed OS vs. Network OS

Feature	Network OS (NOS)	Distributed OS (DOS)
Visibility	User is aware of multiple machines.	User sees one single, powerful machine.
Resource Access	User explicitly logs into remote machine (SSH).	OS automatically migrates processes to free nodes.
Complexity	Low.	Extremely High (data coherence issues).

4. User Interfaces and System Calls

4.1 System Calls: The API to the Kernel

User programs run in **User Mode**. They cannot access hardware directly. To interact with disk, screen, or network, they must switch to **Kernel Mode** via a System Call.

The Mechanism:

1. User program places arguments in registers.

2. Executes a special `TRAP` instruction (software interrupt).
3. Hardware switches mode bit from 1 (User) to 0 (Kernel).
4. OS executes the request (e.g., `read()`).
5. OS executes `Return-from-Trap` , resetting mode bit to 1.

Common POSIX System Calls:

- **Process Control:** `fork()` (create new process), `exit()` (terminate), `wait()` (wait for child).
- **File Management:** `open()` , `read()` , `write()` , `close()` .
- **Information Maintenance:** `getpid()` , `alarm()` , `time()` .

4.2 Worked Example: The `fork()` System Call

In C, `fork()` creates a duplicate process.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    // System Call: Creates a child process
    pid = fork();

    if (pid < 0) {
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) {
        // Child Process
        execlp("/bin/ls", "ls", NULL);
    }
    else {
        // Parent Process
        wait(NULL); // System Call: Wait for child
        printf("Child Complete");
    }
    return 0;
}
```

Analysis for Exam:

- If `fork()` returns 0, you are in the Child process.
- If `fork()` returns a positive integer, you are in the Parent process (the integer is the Child's PID).
- The Child gets a *copy* of the Parent's memory space, not the shared memory (unless explicitly set up).

5. Case Study: The UNIX Philosophy

"Do one thing and do it well."

Unlike monolithic systems that try to do everything, UNIX (and Linux) relies on small, modular utilities combined via **pipes**.

- **Scenario:** Count the number of unique words in a file.
- **Monolithic approach:** Write a complex C++ program with hash maps.
- **UNIX approach:** `cat file.txt | tr ' ' '\n' | sort | uniq | wc -l`
  - `cat` : Read file.
  - `tr` : Translate spaces to newlines (one word per line).
  - `sort` : Sort the list.
  - `uniq` : Remove duplicates.
  - `wc -l` : Count lines.

Exam Relevance: This demonstrates the **Modularity** design goal of operating systems.

6. Unit Summary & Cheat Sheet

Key Differences Table

Feature	Multiprogramming	Multitasking (Time-Sharing)	Multiprocessing
Objective	Maximize CPU utilization.	Maximize Responsiveness.	Increase Throughput.
Mechanism	Switch on I/O wait.	Switch on Timer Interrupt.	Parallel Execution on multiple CPUs.
Context	Batch systems.	Desktops/Laptops.	Servers/Supercomputers.

Diagram Concept: The Dual-Mode Operation

```
User Process  ------------>  System Call (Trap)  ------------>  OS Kernel
(Mode Bit = 1)                                                 (Mode Bit = 0)
                                                                     |
                                                               Execute Request
                                                                     |
Return Result <------------  Return from Trap    <------------  Done
```

7. Exam Strategies & Pitfalls

Common Mistakes

1. **Confusing "Kernel" and "OS":**
    - *Mistake:* Saying "The OS runs the compiler."
    - *Correction:* The compiler is a *system program*, not part of the *kernel*. The kernel allocates memory for the compiler.
2. **Multiprogramming vs Multiprocessing:**
    - *Mistake:* Using them interchangeably.
    - *Correction:* Multiprogramming = One CPU managing multiple jobs. Multiprocessing = Multiple CPUs.
3. **The "Faster" Fallacy:**
    - *Mistake:* "An OS makes the hardware faster."
    - *Correction:* An OS actually adds *overhead* (system calls, context switching). It makes the hardware *easier to use* and *fairer*, but strictly speaking, raw execution is slower than running on bare metal.

Model Answer Tips

- **Keywords:** Always use standard terminology. Don't say "The OS stops the program"; say "The OS *preempts* the process."
- **Diagrams:** If asked about "System Call steps," drawing the transition from User Mode to Kernel Mode (Section 6 diagram) usually guarantees 50% of the marks immediately.

8. Practice Question Bank

Section A: Multiple Choice (1 Mark Each)

Q1. Which component of the OS is resident in the main memory at all times? a) Linker b) Loader c) Kernel d) Shell *Answer: c) Kernel*

Q2. The primary purpose of an Operating System is: a) To make the most efficient use of computer hardware b) To allow people to use the computer c) To keep systems programmers employed d) To make computers fast *Answer: a) or b) depending on context (Mainframe vs PC), but strictly (a) for efficiency.*

Q3. In a time-sharing system, the mechanism used to switch between tasks is: a) Spooling b) Interrupt handling c) Polling d) Compiling *Answer: b) Interrupt handling (specifically Timer Interrupt)*

Section B: Short Answer Questions (5 Marks Each)

Q4. Explain the concept of "Spooling" and how it differs from "Buffering". *Hint:* Focus on Simultaneous Peripheral Operation On-Line. Spooling uses disk as a huge buffer (e.g., printing). Buffering is usually in RAM to smooth out speed mismatches.

Q5. Why is the "Dual-Mode" operation (User/Kernel) essential for system protection? *Solution:* Without dual-mode, a user program could execute privileged instructions (like "halt

machine" or "wipe disk"). By forcing a switch to Kernel mode via a Trap, the OS validates every request before executing it.

Section C: Long Answer / Analytical (10 Marks Each)

Q6. "An Operating System is a resource allocator." Justify this statement with respect to CPU and Memory management. *Structure:*

1. **Introduction:** Define Resource Allocator.
2. **CPU:** Discuss Scheduling (deciding who runs next). Mention Round Robin or Priority.
3. **Memory:** Discuss Allocation (Paging/Segmentation). Preventing processes from overwriting each other.
4. **Conclusion:** The OS acts as the manager to prevent chaos and starvation.

Q7. Modern Operating Systems are "Interrupt Driven." Explain the sequence of events when a user presses a key on the keyboard. *Solution Steps:*

1. Keyboard controller sends an electrical signal (interrupt) to the CPU.
2. CPU finishes current instruction.
3. CPU saves the state (Program Counter, Registers) of the currently running process to the stack/PCB.
4. CPU looks up the Interrupt Vector Table to find the address of the Keyboard Interrupt Service Routine (ISR).
5. CPU jumps to the ISR.
6. ISR reads the key, stores it in a buffer.
7. CPU restores the saved state of the interrupted process and resumes execution.

Q8. (Hard) Calculate the CPU Utilization. *Problem:* A computer has 512 MB of RAM. The OS takes 128 MB. Each user program takes 128 MB and has an 80% average I/O wait time.

1. How many processes can fit in memory?
2. What is the CPU Utilization? *Solution:*
3. Available RAM = $512 - 128 = 384$ MB.
4. Processes $N = 384/128 = 3$ processes.
5. Wait time $p = 0.8$.
6. CPU Utilization = $1 - p^N = 1 - 0.8^3 = 1 - 0.512 = 48.8\%$. *Analysis:* To improve this, we need either more RAM (to increase N) or to reduce I/O wait (p).

Comprehensive Exam Handbook
Module 2: Process Management

Professor (AI) Assistant

Contents

1 Process Management **2**
1.1 Introduction to Module 2 2
1.2 Process Concepts 2
1.2.1 The Process 2
1.2.2 Process State 2
1.2.3 Process Control Block (PCB) 3
1.3 Process Scheduling 3
1.3.1 Schedulers 3
1.3.2 Scheduling Algorithms 3
1.4 Concurrency and Synchronization 4
1.4.1 The Critical Section Problem 4
1.4.2 Peterson's Solution 5
1.4.3 Semaphores 5
1.5 Classical Synchronization Problems 5
1.5.1 Producer-Consumer Problem 5
1.5.2 Dining Philosophers Problem 6
1.6 Deadlocks 6
1.6.1 Four Necessary Conditions 6
1.6.2 Banker's Algorithm (Deadlock Avoidance) 6
1.7 Exam Preparation Guide 8
1.7.1 Common Mistakes to Avoid 8
1.7.2 Practice Questions 8

Chapter 1

Process Management

1.1 Introduction to Module 2

This module forms the backbone of Operating Systems. In the exam, this section usually carries significant weight (often 20-30% of the paper) because it tests both theoretical understanding (definitions, states) and practical problem-solving skills (scheduling algorithms, synchronization, deadlock recovery).

> **Professor's Insight:** Do not just memorize definitions here. The examiners want to see that you understand the *lifecycle* of a process. When a process moves from *Ready* to *Running*, who moves it? The Scheduler. When it moves from *Running* to *Blocked*, why? I/O request. Visualizing this flow is key.

1.2 Process Concepts

1.2.1 The Process

A process is a program in execution. While a program is a passive entity (stored on disk), a process is an active entity.

- **Text Section:** Program code.
- **Program Counter:** Current activity.
- **Stack:** Temporary data (function parameters, return addresses, local variables).
- **Data Section:** Global variables.
- **Heap:** Memory dynamically allocated during run time.

1.2.2 Process State

As a process executes, it changes state.

1. **New:** The process is being created.
2. **Running:** Instructions are being executed.
3. **Waiting (Blocked):** The process is waiting for some event (e.g., I/O completion).
4. **Ready:** The process is waiting to be assigned to a processor.
5. **Terminated:** The process has finished execution.

1.2.3 Process Control Block (PCB)

The OS represents each process using a PCB (also called a Task Control Block). This is a critical data structure.

Attribute	Description
Process State	Current state (New, Ready, etc.)
Program Counter	Address of the next instruction to execute.
CPU Registers	Accumulators, index registers, stack pointers.
CPU Scheduling Info	Priority, pointers to scheduling queues.
Memory Management	Base and limit registers, page tables.
Accounting Info	CPU time used, time limits, process numbers.
I/O Status Info	List of I/O devices allocated, list of open files.

Table 1.1: Components of the PCB

1.3 Process Scheduling

The objective of multiprogramming is to have some process running at all times to maximize CPU utilization.

1.3.1 Schedulers

- **Long-Term Scheduler (Job Scheduler):** Selects processes from the storage pool and loads them into memory. Controls the *degree of multiprogramming*.
- **Short-Term Scheduler (CPU Scheduler):** Selects from among the processes that are ready to execute and allocates the CPU to one of them. Executes very frequently (milliseconds).
- **Medium-Term Scheduler:** Handles *swapping*. Removes processes from memory to reduce the degree of multiprogramming and later reintroduces them.

1.3.2 Scheduling Algorithms

This is the most common area for numerical questions in exams.

Key Metrics

- **Burst Time (BT):** Time required by a process for CPU execution.
- **Arrival Time (AT):** Time at which the process enters the ready queue.
- **Turnaround Time (TAT):** Completion Time - Arrival Time.
- **Waiting Time (WT):** Turnaround Time - Burst Time.

Worked Example 1: FCFS (First-Come, First-Served)

Problem: Consider the following processes:

Process	Arrival Time	Burst Time
P1	0	24
P2	1	3
P3	2	3

Solution: Gantt Chart:

[P1 (0-24)] [P2 (24-27)] [P3 (27-30)]

Calculations:

- **P1:** Completion=24, TAT = $24 - 0 = 24$, WT = $24 - 24 = 0$.
- **P2:** Completion=27, TAT = $27 - 1 = 26$, WT = $26 - 3 = 23$.
- **P3:** Completion=30, TAT = $30 - 2 = 28$, WT = $28 - 3 = 25$.
- **Average WT:** $(0 + 23 + 25)/3 = 16$ ms.

Worked Example 2: SJF (Shortest Job First) - Non-Preemptive

Problem:

Process	Arrival Time	Burst Time
P1	0	6
P2	1	8
P3	2	7
P4	3	3

Solution: 1. At t=0, only P1 is present. P1 runs (0-6). 2. At t=6, P2, P3, P4 are all present. Burst times: P2(8), P3(7), P4(3). P4 is shortest. P4 runs (6-9). 3. At t=9, P2(8) and P3(7) remain. P3 is shortest. P3 runs (9-16). 4. P2 runs (16-24).

Gantt Chart:

[P1 (0-6)] [P4 (6-9)] [P3 (9-16)] [P2 (16-24)]

Calculations (Waiting Time):

- P1: $0 - 0 = 0$
- P4: $6 - 3 = 3$
- P3: $9 - 2 = 7$
- P2: $16 - 1 = 15$
- Average WT: $(0 + 3 + 7 + 15)/4 = 6.25$ ms.

1.4 Concurrency and Synchronization

1.4.1 The Critical Section Problem

When multiple processes access shared data concurrently, data inconsistency can occur. To ensure consistency, we need synchronization. **Critical Section (CS):** A segment of code where the process accesses shared resources.

Requirements for a Solution: 1. **Mutual Exclusion:** If process Pi is in its CS, no other processes can be in their CS. 2. **Progress:** If no process is in CS and some wish to enter, only those not in their remainder section can participate in the decision. 3. **Bounded Waiting:** There must be a limit on the number of times other processes are allowed to enter their CS after a process has made a request.

1.4.2 Peterson's Solution

A software-based solution for two processes. It uses two variables:

```
int turn;
boolean flag[2];

// Structure for Process i
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j); // Busy wait

        /* CRITICAL SECTION */

    flag[i] = false;

        /* REMAINDER SECTION */
} while (true);
```

1.4.3 Semaphores

A semaphore S is an integer variable accessed only through two atomic operations: `wait()` (or P) and `signal()` (or V).

- **Counting Semaphore:** Integer value can range over an unrestricted domain.
- **Binary Semaphore (Mutex):** Integer value can range only between 0 and 1.

```
wait(S) {
    while (S <= 0); // Busy wait
    S--;
}

signal(S) {
    S++;
}
```

1.5 Classical Synchronization Problems

1.5.1 Producer-Consumer Problem

Goal: A producer creates items and puts them in a buffer; a consumer takes them out. They must not overwrite or read empty slots.

Solution using Semaphores:

- `mutex` = 1 (Binary, protects buffer)

- `empty` = n (Counting, tracks empty slots)
- `full` = 0 (Counting, tracks filled slots)

```
// Producer
do {
    // produce an item
    wait(empty);
    wait(mutex);
    // add item to buffer
    signal(mutex);
    signal(full);
} while(true);

// Consumer
do {
    wait(full);
    wait(mutex);
    // remove item from buffer
    signal(mutex);
    signal(empty);
    // consume the item
} while(true);
```

1.5.2 Dining Philosophers Problem

Five philosophers sit at a table. There is one chopstick between each pair. A philosopher needs two chopsticks to eat. **Challenge:** Avoid deadlock (everyone picks up left chopstick and waits for right) and starvation.

1.6 Deadlocks

A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

1.6.1 Four Necessary Conditions

Deadlock can arise if **all four** hold simultaneously: 1. **Mutual Exclusion:** At least one resource is non-sharable. 2. **Hold and Wait:** A process holding at least one resource is waiting to acquire additional resources held by others. 3. **No Preemption:** Resources cannot be taken away voluntarily. 4. **Circular Wait:** P_0 waits for P_1, P_1 waits for P_2, ..., P_n waits for P_0.

1.6.2 Banker's Algorithm (Deadlock Avoidance)

This algorithm checks if allocating resources leaves the system in a **Safe State**.

Worked Example 3: Banker's Algorithm

System State: 5 Processes (P_0 to P_4), 3 Resource Types (A, B, C). Available: [3, 3, 2]

Process	Allocation			Max		
	A	B	C	A	B	C
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

Step 1: Calculate Need Matrix (Max - Allocation)

- P0: $753 - 010 = 743$
- P1: $322 - 200 = 122$
- P2: $902 - 302 = 600$
- P3: $222 - 211 = 011$
- P4: $433 - 002 = 431$

Step 2: Find Safe Sequence Current Work (Available) = [3, 3, 2]
1. Check P0: Need(7,4,3) $\leq$ Work(3,3,2)? False. 2. Check P1: Need(1,2,2) $\leq$ Work(3,3,2)? True.

- Execute P1.
- New Work = Work + Allocation(P1) = $332 + 200 = 532$.

3. Check P3: Need(0,1,1) $\leq$ Work(5,3,2)? True.

- Execute P3.
- New Work = $532 + 211 = 743$.

4. Check P4: Need(4,3,1) $\leq$ Work(7,4,3)? True.

- Execute P4.
- New Work = $743 + 002 = 745$.

5. Check P0: Need(7,4,3) $\leq$ Work(7,4,5)? True.

- Execute P0.
- New Work = $745 + 010 = 755$.

6. Check P2: Need(6,0,0) $\leq$ Work(7,5,5)? True.

- Execute P2.

Safe Sequence: $< P1, P3, P4, P0, P2 >$. System is Safe.

1.7 Exam Preparation Guide

1.7.1 Common Mistakes to Avoid

- **Gantt Chart Errors:** In Round Robin, forgetting to add a process back to the queue after its time quantum expires.
- **Context Switch Overhead:** Assuming context switching is instantaneous. If the question gives a switch time (e.g., Δ), you must subtract it from useful CPU time.
- **Preemptive vs Non-Preemptive:** Misreading the question. In SJF, preemptive is called SRTF (Shortest Remaining Time First). The results are totally different.
- **Deadlock Definition:** Confusing "Starvation" with "Deadlock". Starvation is indefinite waiting; Deadlock is a circular wait where nothing moves.

1.7.2 Practice Questions

Multiple Choice

1. Which module gives control of the CPU to the process selected by the short-term scheduler?

(a) Dispatcher

(b) Interrupt

(c) Scheduler

(d) None of the above

Answer: (a) Dispatcher

2. The circular wait condition can be prevented by:

(a) Using a timer

(b) Defining a linear ordering of resource types

(c) Using semaphores

(d) All of the above

Answer: (b)

Long Answer Question

Q: Explain the Producer-Consumer problem. Why is a mutex required along with full/empty semaphores?

Hint for Solution: Start by defining the problem (shared buffer). Explain the race condition if two pointers are updated simultaneously. Provide the semaphore solution code.

- **Why Mutex?** The 'count++' and 'count–' operations in high-level languages are not atomic (they compile to 3 assembly instructions: Load, Inc/Dec, Store). If a context switch happens in the middle, the count becomes corrupt. The Mutex ensures that only one process manipulates the buffer pointers/count at a time.

Module 3: Memory Management

Estimated Study Time: 12 Hours **Unit Weightage:** High (Critical for System Performance questions)

1. Introduction: The Reality of Memory

In an ideal world, memory would be infinitely large, infinitely fast, and non-volatile. In reality, we are stuck with a hierarchy where fast memory (Registers/Cache) is expensive and small, and cheap memory (Disk) is painfully slow.

Memory Management is the art of managing this hierarchy to give processes the illusion of abundant, fast memory while efficiently sharing the limited physical RAM.

1.1 The Memory Hierarchy

The CPU is fast. Memory is slow. If the CPU had to wait for main memory (RAM) for every instruction, it would spend most of its time idle.

The Hierarchy Pyramid:

1. **Registers:** Inside CPU. Zero latency. (< 1 ns)
2. **Cache (L1, L2, L3):** SRAM. Very fast. (1-10 ns)
3. **Main Memory (RAM):** DRAM. Volatile. (10-100 ns)
4. **Secondary Storage (SSD/HDD):** Persistent. Slow. (Microseconds to Milliseconds)

Key Concept: Locality of Reference This is the only reason the hierarchy works.

- **Temporal Locality:** If you access data now, you will likely access it again soon (e.g., loop variables).
- **Spatial Locality:** If you access address X, you will likely access $X + 1$ soon (e.g., arrays).

2. Memory Allocation Strategies

How do we fit multiple processes into memory at once?

2.1 Contiguous Memory Allocation

The old school method. Each process gets one single continuous block of memory.

A. Fixed Partitioning (Multiprogramming with Fixed Tasks - MFT)

Memory is divided into fixed-sized partitions (e.g., 4 blocks of 100MB).

- **Pros:** Simple implementation.
- **Cons: Internal Fragmentation**. If a process needs 70MB and goes into a 100MB block, 30MB is wasted *inside* the allocated block.

B. Variable Partitioning (Multiprogramming with Variable Tasks - MVT)

Memory is allocated exactly as much as the process needs.

- **Pros:** No internal fragmentation.
- **Cons: External Fragmentation**. As processes enter and leave, small holes of free memory are created. These holes might total 100MB, but if they are not contiguous, we cannot load a 100MB process.
    - *Solution:* **Compaction** (Shuffling memory to consolidate free space). Expensive and freezes the system.

C. Allocation Algorithms (The "Fitting" Problems)

Given a set of holes, where do we put a new process?

1. **First Fit:** Allocate the first hole that is big enough. (Fastest).
2. **Best Fit:** Allocate the smallest hole that is big enough. (Creates tiny, useless holes).
3. **Worst Fit:** Allocate the largest hole available. (Counter-intuitive, but leaves larger remaining holes).

    Worked Example 1: Allocation Algorithms

    Scenario: Memory partitions (holes) are: 100 KB, 500 KB, 200 KB, 300 KB, 600 KB (in order). **Processes arrive:** P1 (212 KB), P2 (417 KB), P3 (112 KB), P4 (426 KB).

    Task: Apply **First Fit**.

    1. **P1 (212 KB):** Skips 100. Skips 500? No, 500 > 212. Goes into 500 KB. (Hole remaining: 288 KB).
    2. **P2 (417 KB):** Skips 100. Skips 288 (leftover). Skips 200. Skips 300. Goes into 600 KB. (Hole remaining: 183 KB).
    3. **P3 (112 KB):** Skips 100. Checks 288 (leftover from P1). Fits! (Hole remaining: 176 KB).
    4. **P4 (426 KB):** Needs 426. Available: 100, 176, 200, 300, 183. None big enough. **P4 Waits.**

3. Non-Contiguous Allocation: Paging

Paging eliminates external fragmentation by allowing a process to be scattered across memory.

3.1 The Concept

- **Physical Memory** is broken into fixed-sized blocks called **Frames**.
- **Logical Memory** (Process view) is broken into same-sized blocks called **Pages**.
- **The Page Table:** The map that translates "Page 1" to "Frame 5".

3.2 Address Translation

The CPU generates a **Logical Address**:

$$\text{Address} =< \text{Page Number (p)}, \text{Offset (d)} >$$

The MMU (Memory Management Unit) uses the Page Table to find the physical frame (f).

$$\text{Physical Address} = (f \times \text{Frame Size}) + d$$

Worked Example 2: Paging Calculation

Given:

- Logical Address Space: 2^{16} bytes (64 KB)
- Page Size: 4 KB (2^{12} bytes)
- Physical Memory: 32 KB

Question: How many bits for Page Number (p) and Offset (d)?

Solution:

1. **Offset (d):** Page size is 2^{12}. We need **12 bits** to address inside the page.
2. **Page Number (p):** Logical address is 16 bits total.

$$p = \text{Total Bits} - d = 16 - 12 = 4 \text{ bits}$$

So, the process can have $2^4 = 16$ pages.

4. Virtual Memory & Demand Paging

"Virtual Memory is a technique that allows the execution of processes that are not completely in memory."

4.1 Concept

We only load the pages we actually need (*Demand Paging*).

- **Page Fault:** When the CPU tries to access a page that is not in RAM (Invalid bit in Page Table).
- **Swapping:** Moving pages between RAM and Disk (Backing Store).

4.2 Page Replacement Algorithms

When RAM is full and a Page Fault occurs, we must evict a victim page. Which one?

A. FIFO (First-In, First-Out)

Evict the oldest page.

- **Issue: Belady's Anomaly**. Increasing the number of frames can sometimes *increase* the number of page faults.

B. Optimal (OPT)

Evict the page that will not be used for the longest period of time.

- **Issue:** Impossible to implement because we can't predict the future. Used only as a benchmark.

C. LRU (Least Recently Used)

Evict the page that hasn't been used for the longest time.

- **Theory:** Based on temporal locality. If you haven't used it recently, you probably won't use it soon.
- **Implementation:** Requires hardware support (counters or stacks) which can be expensive.

Worked Example 3: LRU vs. FIFO

Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 **Frames:** 3

LRU Walkthrough:

1. Load 7, 0, 1 (Faults: 3). Frame: [7, 0, 1]
2. Access 2. Evict 7 (Oldest usage). Frame: [2, 0, 1]. (Fault: 4)
3. Access 0. Hit. (0 is now most recent). Frame: [2, 0, 1]
4. Access 3. Evict 1 (LRU). Frame: [2, 0, 3]. (Fault: 5)
5. Access 0. Hit. Frame: [2, 0, 3]
6. Access 4. Evict 2 (LRU). Frame: [4, 0, 3]. (Fault: 6) ... and so on.

5. Exam Strategies & Common Mistakes

Common Pitfalls

1. **Confusing Sizes:** Mixing up bits and bytes. If asked for address *lines*, the answer is bits (e.g., 16 bits). If asked for address *space*, the answer is bytes (e.g., 64 KB).
2. **Offset Invariance:** The offset (d) never changes during translation. It is copied directly from logical to physical address. If you calculate a new offset, you are wrong.
3. **TLB Confusion:** A TLB Miss does NOT necessarily mean a Page Fault. You can miss the TLB (cache of page table) but find the page in RAM.

Tips for High Scores

- **Draw Diagrams:** For Paging questions, draw the box diagram: CPU -> Logical Address -> Page Table -> Physical Address -> Memory.
- **Show Steps:** In Replacement Algorithms, write out the state of the frames at *every* step. One small slip ruins the final count.
- **Keywords:** Use terms like "Internal Fragmentation," "Thrashing," "Locality of Reference," and "Context Switch."

6. Unit Summary & Cheat Sheet

Concept	Definition	Key Formula/Characteristic
Logical Address	Generated by CPU	$2^{\text{Logical Address Bits}}$
Physical Address	Actual location in RAM	$2^{\text{Physical Address Bits}}$
Page Size	Size of block in Logic/Physical	determines Offset (d) bits
Internal Frag.	Wasted space *inside* block	Happens in Fixed Partitioning & Paging
External Frag.	Wasted space *between* blocks	Happens in Variable Partitioning
Thrashing	High paging activity	CPU utilization drops to near zero
TLB	Translation Lookaside Buffer	Fast cache for Page Table entries

7. Practice Questions

Section A: Multiple Choice

1. **Which fragmentation does paging suffer from?** a) External Fragmentation b) Internal Fragmentation c) Both d) Neither *Answer: (b). The last page of a process may not fill the frame completely.*
2. **In which page replacement policy does Belady's Anomaly occur?** a) LRU b) Optimal c) FIFO d) Random *Answer: (c). Adding frames can actually cause more faults in FIFO.*

Section B: Short Answer

1. **Explain the difference between Logical and Physical addresses.** *Hint: Focus on CPU view vs. Memory Bus view and the role of the MMU.*
2. **What is Thrashing? How does the OS detect it?** *Hint: Thrashing is when the system spends more time paging than executing. Detected by low CPU utilization combined with high disk activity.*

Section C: Long Answer / Calculation

Q1: A system has a 32-bit logical address space and a 4 KB page size. The Page Table Entry (PTE) size is 4 Bytes. a) How many entries are in the Page Table? b) What is the size of the Page Table in MB?

```
*Solution Hint:*
* Page size = 4 KB = $2^{12}$.
* Address bits = 32.
* Page Number bits = $32 - 12 = 20$.
* Entries = $2^{20} = 1$ Million.
```

```
* Size = $1 \text{ Million} \times 4 \text{ Bytes} = 4 \text{ MB}$.
```

8. Case Study: The "Out of Memory" Killer (Linux)

In modern Linux systems, memory is over-committed. The OS promises more memory to processes than physically exists, betting that they won't all use it at once.

The Problem: What happens when they *do* use it? **The Solution:** The **OOM (Out of Memory) Killer**. This is a kernel process that reviews all running processes and assigns them a "badness score" based on how much RAM they use and how long they've been running. When RAM is critically low, the OOM Killer brutally terminates the process with the highest score to free up space instantly.

Real World Application: This is why your Android phone closes background apps when you launch a heavy game. It's not a bug; it's a ruthless memory management strategy.

Module 4: File Systems and Input/Output Systems

Estimated Study Time: 8 Hours **Weight:** High (Critical for understanding OS-Hardware interaction)

1. Unit Overview & Learning Objectives

This module bridges the gap between the logical view of data (files, directories) and the physical reality of storage hardware (disks, blocks, signals).

The Hard Truth: Most students fail questions here because they cannot distinguish between *File System Implementation* (inodes, FAT) and *File Allocation* (how blocks are reserved). Do not mix them up.

Key Topics:

- **File Logic:** Attributes, Operations, Types.
- **File Physics:** Allocation methods (Contiguous, Linked, Indexed).
- **Directory Logic:** Single-level, Two-level, Tree, Acyclic Graph.
- **I/O Systems:** Polling, Interrupts, DMA, Scheduling.

2. File System Organization

2.1 File Attributes & Control Blocks

A "file" is an abstraction. To the OS, a file is a collection of related blocks. The OS maintains a structure called the **File Control Block (FCB)** (or Inode in Unix) to manage this.

Standard Attributes:

1. **Name:** The only information kept in human-readable form.
2. **Identifier:** A unique tag (usually a number) identifies the file within the file system; it is the non-human-readable name for the file.
3. **Type:** Needed for systems that support different types (e.g., .exe, .txt).
4. **Location:** Pointer to a device and to the location of the file on that device.
5. **Size:** Current file size (in bytes, words, or blocks).
6. **Protection:** Access-control information determines who can do reading, writing, executing, and so on.
7. **Time, date, and user identification:** Data for protection, security, and usage monitoring.

2.2 File Operations

The OS provides system calls to create, write, read, reposition, delete, and truncate files.

- **Open(Fi):** The most critical operation. It does not "open" a physical door. It searches the directory structure on disk for entry F_i and moves the content of entry to memory.
    - *Result:* Returns a file handle (pointer) to the Open File Table in memory.
- **Close(Fi):** Moves the content of the Open File Table entry to the directory structure on disk.

Real-World Context: When you double-click a file in Windows, the OS performs an `open()` system call. If you delete a file while it is open, Unix systems remove the directory entry but keep the inode active until the file is closed. Windows typically locks the file and prevents deletion.

3. File Allocation Methods (CRITICAL EXAM TOPIC)

This is the most common area for numerical problems. You must understand how disk blocks are assigned.

3.1 Contiguous Allocation

Each file occupies a set of contiguous blocks on the disk.

- **Pros:** Best performance for sequential and direct access. Simple (only start block & length needed).
- **Cons:** External fragmentation. File cannot grow if no adjacent space is free.
- **Analogy:** Storing a book on a shelf where all pages must be next to each other.

3.2 Linked Allocation

Each file is a linked list of disk blocks. The directory contains a pointer to the first and last blocks.

- **Pros:** No external fragmentation. File size can grow dynamically.
- **Cons:** Slow for direct access (must traverse the list). Reliability (if a pointer is lost, file is corrupted).
- **Analogy:** A scavenger hunt where each clue points to the next.

3.3 Indexed Allocation

Brings all pointers together into one block: the **Index Block**.

- **Pros:** Supports direct access without external fragmentation.
- **Cons:** Wasted space for index blocks (overhead).
- **Implementation:** Unix Inodes use a hybrid approach (Direct pointers + Single Indirect + Double Indirect + Triple Indirect).

Worked Example 1: Calculating Maximum File Size (Indexed Allocation)

Problem: A file system uses 4KB blocks and 4-byte block pointers. It uses an index node with 12 direct pointers, 1 single indirect pointer, and 1 double indirect pointer. What is the maximum file size?

Solution:

1. **Block Size:** $4KB = 4096$ bytes.
2. **Pointer Size:** 4 bytes.
3. **Pointers per block:** $4096/4 = 1024$ pointers.

- **Direct Pointers:** $12 \times 4KB = 48KB$.
- **Single Indirect:** Points to 1 block holding 1024 pointers.
  - Capacity: $1024 \times 4KB = 4MB$.
- **Double Indirect:** Points to 1 block holding 1024 pointers, each pointing to another block of 1024 pointers.
  - Capacity: $1024 \times 1024 \times 4KB = 1024 \times 4MB = 4GB$.

Total Max Size: $48KB + 4MB + 4GB \approx 4GB$.

4. Directory Structure

Directories are symbol tables that translate file names into their directory entries.

Structure	Description	Pros	Cons
Single-Level	All files in one directory.	Simple.	Naming collisions; hard to manage with many users.
Two-Level	Master File Directory (MFD) -> User File Directory (UFD).	User isolation; no name collision between users.	No grouping capability for users (subdirectories).
Tree-Structured	Standard modern approach. Arbitrary depth.	Efficient grouping; logical.	Navigation complexity.
Acyclic Graph	Allows shared subdirectories/files (aliasing/links).	flexible sharing.	Danger of dangling pointers; multiple traversal paths.

5. Input/Output Systems

5.1 Hardware Concepts

- **Port:** Connection point (e.g., USB).
- **Bus:** Shared wires (PCIe, SCSI).
- **Controller:** Electronics that operate the port/bus/device.

5.2 I/O Communication Techniques (The "Big Three")

1. **Programmed I/O (Polling):** CPU constantly checks a status bit on the device controller ("Are you ready?"). Wastes CPU cycles.

2. **Interrupt-Driven I/O:** Device sends a signal (interrupt) when ready. CPU switches context to Interrupt Handler. Efficient for slow devices (keyboard).
3. **Direct Memory Access (DMA):** Used for bulk data (disk/network). CPU programs the DMA controller and goes back to work. DMA transfers data directly to RAM. CPU is interrupted only when the *entire block* is done.

6. Disk Scheduling Algorithms (HIGH PROBABILITY EXAM TOPIC)

The goal is to minimize **Seek Time** (movement of the disk arm).

Scenario:

- **Head Start:** 53
- **Queue:** 98, 183, 37, 122, 14, 124, 65, 67
- **Range:** 0 to 199

1. FCFS (First-Come, First-Served)

Process requests in order.

- Order: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
- **Total Head Movement:** $|98 - 53| + |183 - 98| + ... = 640$ cylinders.
- *Verdict:* Simple but inefficient.

2. SSTF (Shortest Seek Time First)

Select request closest to current head position.

- Start at 53.
- Closest is 65. (Move: 12)
- Closest to 65 is 67. (Move: 2)
- Closest to 67 is 37. (Move: 30)
- Order: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
- **Total Head Movement:** 236 cylinders.
- *Verdict:* Drastic improvement over FCFS, but potential for starvation of distant requests.

3. SCAN (Elevator Algorithm)

Arm moves in one direction (e.g., towards 0), servicing requests, hits the end, and reverses.

- Assume moving towards 0.
- Order: 53 -> 37 -> 14 -> 0 (Hit edge!) -> 65 -> 67 -> 98 -> 122 -> 124 -> 183
- *Verdict:* Good distribution, no starvation.

4. C-SCAN (Circular SCAN)

Like SCAN, but when it hits the end, it strictly returns to the beginning without servicing requests on the return trip.

- *Verdict:* Provides more uniform wait time.

7. Exam Strategy & Common Mistakes

Common Mistakes to Avoid

1. **Confusing "Logical Address" with "Physical Address" in Files:** Do not confuse the offset within a file (Logical) with the block number on the disk (Physical).
2. **Scheduling Calculation Errors:** In SCAN/C-SCAN, students often forget that the head must go *all the way to the edge* (0 or 199) unless the question specifies "LOOK" scheduling.
    - *Rule:* SCAN touches the edge. LOOK only goes to the last request.
3. **Inode Math:** Failing to convert units (e.g., Bytes to Kilobytes) when calculating maximum file sizes.

Model Answer Tip

When asked to "Explain the file system layers," do not just list them. Draw the diagram: `Application Programs -> Logical File System -> File-Organization Module -> Basic File System -> I/O Control -> Devices`.

- **Logical FS:** Manages Metadata (inodes).
- **File-Org Module:** Maps logical blocks to physical blocks.
- **Basic FS:** Issues generic commands to driver (read block 123).

8. Practice Bank

Section A: Multiple Choice (1 Mark)

1. **Which allocation method suffers from external fragmentation?** a) Linked b) Indexed c) Contiguous d) Hashed *Answer: c*
2. **In a Unix-style inode, where is the location of the file data stored if the file is very small?** a) In the directory entry b) In the direct pointers of the inode c) In the Superblock d) In the boot block *Answer: b*
3. **Which structure allows a file to have multiple parent directories?** a) Tree b) Acyclic Graph c) Single Level d) Stack *Answer: b*

Section B: Short Answer (5 Marks)

Q: Explain the difference between blocking and non-blocking I/O.

- **Blocking I/O:** The process is moved to the wait queue until the I/O operation completes. It creates a simple programming model but halts execution.

- **Non-blocking I/O:** The call returns immediately with whatever data is available (or an error code). The process continues execution. Useful for user interfaces (keyboard/mouse) where you can't freeze the screen waiting for a keypress.

Section C: Long Answer / Design (10 Marks)

Q: Compare and contrast Polling, Interrupts, and DMA. Under what system load is polling actually better than interrupts?

- **Structure your answer:**
  - Define all three.
  - **Polling:** CPU busy-wait. Good for very fast devices where the wait is shorter than the context-switch overhead of an interrupt (e.g., a high-speed status register).
  - **Interrupts:** CPU notified on completion. Good for moderate speed devices (Keyboards, network packets). Overhead is the context switch.
  - **DMA:** CPU offloads transfer. Essential for high-throughput (Disk, GPU).
  - **Crucial Point:** Polling is strictly better than interrupts if the device is fast enough that the time to poll is less than the time to push/pop the stack for an interrupt handler.

9. Unit Summary Cheat Sheet

Concept	Key Characteristic	Best Use Case
Contiguous Allocation	Fast, simple, suffers external frag.	Read-only media (CD-ROM).
Linked Allocation	No external frag, slow access.	Sequential files (Logs).
Indexed Allocation	Fast access, pointer overhead.	General purpose FS (Linux).
DMA	Bypasses CPU for memory ops.	Hard Drives, SSDs.